Experiment No:01

Title: To develop a program and simulate the Pulse Amplitude Modulation (PAM) signal and analyze its waveform.

Objective:

- To develop a program to generate a Pulse Amplitude Modulated (PAM) signal
- To simulate the PAM waveform using Python
- To analyze the characteristics of the PAM signal

Theory:

Pulse Amplitude Modulation (PAM) is a modulation technique in which the amplitude of a pulse carrier is varied according to the instantaneous amplitude of the message (modulating) signal.

In PAM:

- A continuous-time message signal is sampled at regular intervals
- Each sample is represented by a pulse

There are mainly two types of PAM:

1. **Natural PAM** – Pulse follows the exact shape of the signal
2. **Flat-top PAM** – Pulse amplitude remains constant during the pulse duration

Mathematically, PAM can be expressed as:

$$s(t) = m(t) \cdot p(t)$$

Where:

- $m(t)$= Message signal
- $p(t)$= Pulse train

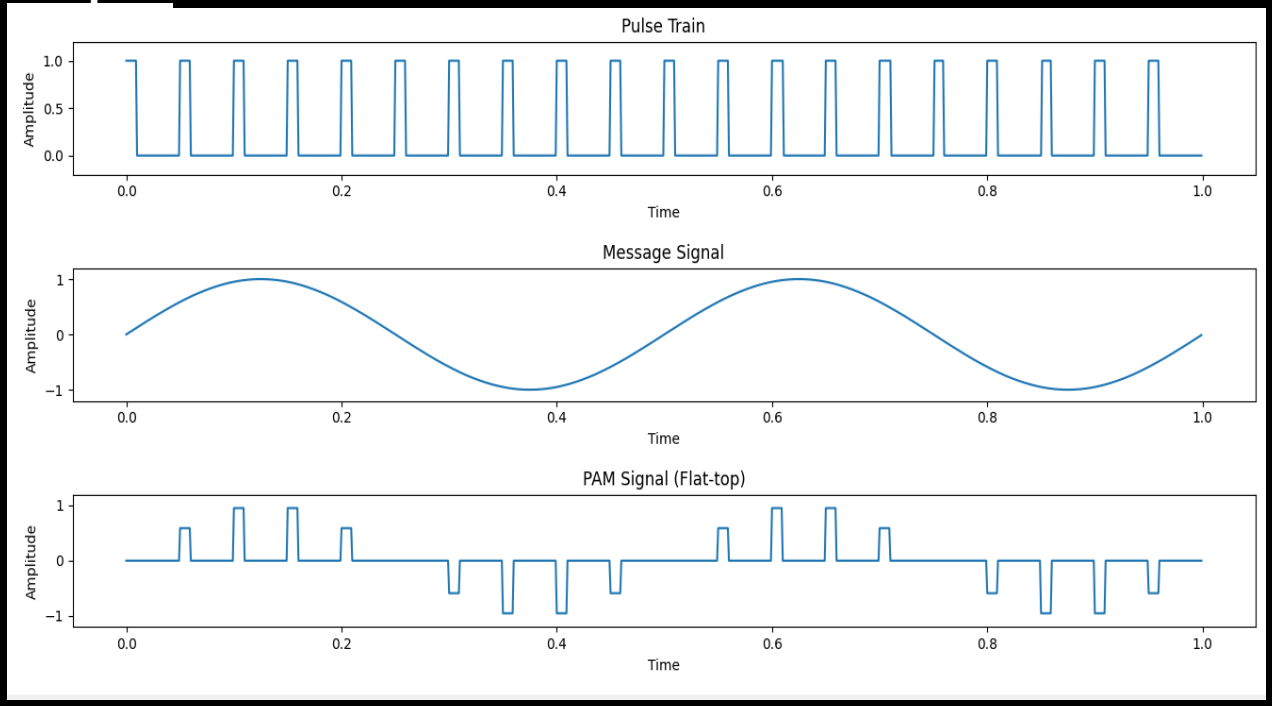
Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import square
fc = 20       # carrier frequency
fm = 2        # message frequency
fs = 1000     # sampling frequency
```

```
t = 1          # duration
n = np.arange(0, t, 1/fs)
duty = 20
s = square(2 * np.pi * fc * n, duty/100)
s[s < 0] = 0   # make it unipolar
m = np.sin(2 * np.pi * fm * n)
period_samp = int(len(n) / fc)
ind = np.arange(0, len(n), period_samp)
on_samp = int(np.ceil(period_samp * duty / 100))
pam = np.zeros(len(n))
for i in ind:
    end = min(i + on_samp, len(n))  # avoid overflow
    pam[i:end] = m[i]
plt.figure(figsize=(10, 6))
plt.subplot(3,1,1)
plt.plot(n, s)
plt.title("Pulse Train")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.ylim([-0.2, 1.2])
plt.subplot(3,1,2)
plt.plot(n, m)
plt.title("Message Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.ylim([-1.2, 1.2])
plt.subplot(3,1,3)
plt.plot(n, pam)
plt.title("PAM Signal (Flat-top)")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.ylim([-1.2, 1.2])
```

```
plt.tight_layout()
plt.show()
```

Output:



Result and Discussion:

From the simulation, three waveforms are observed:

1. **Message Signal**
   - A low-frequency sine wave
   - Represents the original information signal
2. **Pulse Train**
   - High-frequency square pulses
   - Acts as a sampling signal
3. **PAM Signal**
   - The amplitude of pulses varies according to the message signal
   - Clearly shows sampled version of the original signal

Analysis

- The PAM signal successfully represents the original signal in discrete form
- Higher sampling frequency results in better signal representation

Experiment No:-02

Title: To design a program and simulate the Amplitude Shift Keying (ASK) modulation technique.

Objective:

- To understand the basic concept of Amplitude Shift Keying (ASK).
- To design a program that generates ASK modulated signals.
- To simulate and analyze the waveform of ASK modulation.

Theory:

Amplitude Shift Keying (ASK) is a digital modulation technique in which the amplitude of a carrier signal is varied according to the digital input signal (binary data), while frequency and phase remain constant.

- For binary **1** → carrier signal is transmitted with full amplitude
- For binary **0** → carrier signal is either reduced or made zero

Mathematically, ASK signal can be represented as:

$$s(t) = A_c \cdot m(t) \cdot \cos(2\pi f_c t)$$

Where:

- A_c= Carrier amplitude
- f_c= Carrier frequency
- $m(t)$= Message signal (binary data: 0 or 1)

Source Code :

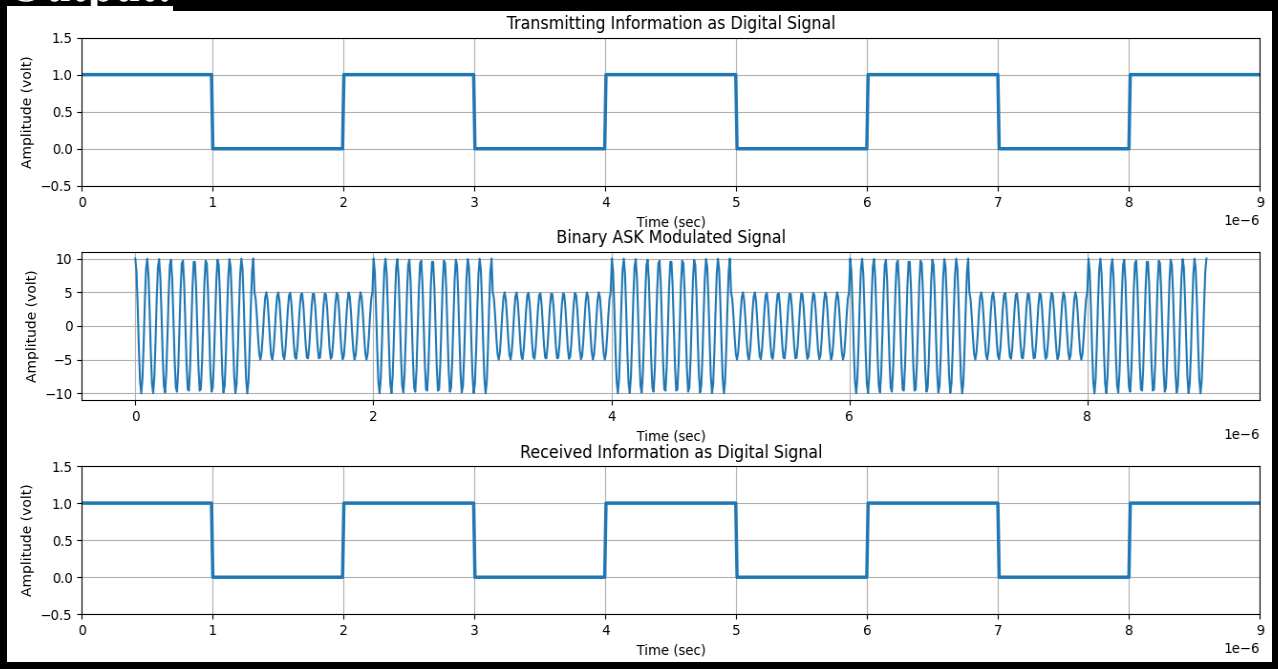
```
import numpy as np
import matplotlib.pyplot as plt
x = [1, 0, 1, 0, 1, 0, 1, 0, 1]
print("Binary Information at transmitter:", x)
bp = 1e-6   # bit period
samples = 100
bit = []
for b in x:
    if b == 1:
```

```
        se = np.ones(samples)
    else:
        se = np.zeros(samples)
    bit.extend(se)
bit = np.array(bit)
t1 = np.linspace(0, bp*len(x), len(bit))
plt.figure(figsize=(10, 8))
plt.subplot(3, 1, 1)
plt.plot(t1, bit, linewidth=2.5)
plt.grid(True)
plt.ylim(-0.5, 1.5)
plt.xlim(0, bp*len(x))
plt.ylabel('Amplitude (volt)')
plt.xlabel('Time (sec)')
plt.title('Transmitting Information as Digital Signal')
A1 = 10
A2 = 5
br = 1 / bp
f = br * 10
t2 = np.linspace(0, bp, samples)
ss = len(t2)
m = []
for b in x:
    if b == 1:
        y = A1 * np.cos(2 * np.pi * f * t2)
    else:
        y = A2 * np.cos(2 * np.pi * f * t2)
    m.extend(y)
m = np.array(m)
t3 = np.linspace(0, bp*len(x), len(m))
plt.subplot(3, 1, 2)
plt.plot(t3, m)
```

```
plt.grid(True)
plt.xlabel('Time (sec)')
plt.ylabel('Amplitude (volt)')
plt.title('Binary ASK Modulated Signal')
mn = []
for i in range(0, len(m), ss):
    segment = m[i:i+ss]
    t = np.linspace(0, bp, ss)
    carrier = np.cos(2 * np.pi * f * t)
    product = segment * carrier
    z = np.trapz(product, t)
    zz = (2 * z / bp)
    threshold = (A1 + A2) / 2
    if zz > threshold:
        mn.append(1)
    else:
        mn.append(0)
print("Binary Information at Receiver:", mn)
bit_rx = []
for b in mn:
    if b == 1:
        se = np.ones(samples)
    else:
        se = np.zeros(samples)
    bit_rx.extend(se)
bit_rx = np.array(bit_rx)
t4 = np.linspace(0, bp*len(mn), len(bit_rx))
plt.subplot(3, 1, 3)
plt.plot(t4, bit_rx, linewidth=2.5)
plt.grid(True)
plt.ylim(-0.5, 1.5)
plt.xlim(0, bp*len(mn))
```

```
plt.xlabel('Time (sec)')
plt.ylabel('Amplitude (volt)')
plt.title('Received Information as Digital Signal')
plt.tight_layout()
plt.show()
```

Output:



Result and Discussion:

In this experiment, the ASK modulation technique was successfully implemented using Python. Three signals were generated and observed:

1. **Message Signal:**
   A digital binary sequence (e.g., 1 0 1 1 0 1) represented as a step waveform.
2. **Carrier Signal:**
   A high-frequency cosine wave that serves as the transmission signal.
3. **ASK Modulated Signal:**
   - When the input bit is **1**, the carrier signal appears with full amplitude.
   - When the input bit is **0**, the signal becomes zero (no carrier)

Experiment No:-03

Title: To construct a program and simulate the Phase Shift Keying (PSK) modulation technique.

Objective:

- To construct a program for simulating Phase Shift Keying (PSK) modulation.
- To analyze the waveform of the PSK signal.
- To understand how digital data is transmitted using phase variations of a carrier signal.

Theory:

Phase Shift Keying (PSK) is a digital modulation technique in which the phase of a carrier signal is varied according to the digital input signal.

In Binary Phase Shift Keying (BPSK), there are two phases:

- 0° phase represents binary '1'
- 180° phase represents binary '0'

The general PSK signal can be expressed as:

$$s(t) = A\cos(2\pi f_c t + \phi)$$

Where:

- A= Amplitude of the carrier signal
- f_c= Carrier frequency
- ϕ= Phase shift (0 or π) depending on the data

In BPSK:

- For bit 1 $\rightarrow \phi = 0$
- For bit 0 $\rightarrow \phi = \pi$

Thus, the phase of the carrier flips by 180° whenever the bit changes.

Source Code :

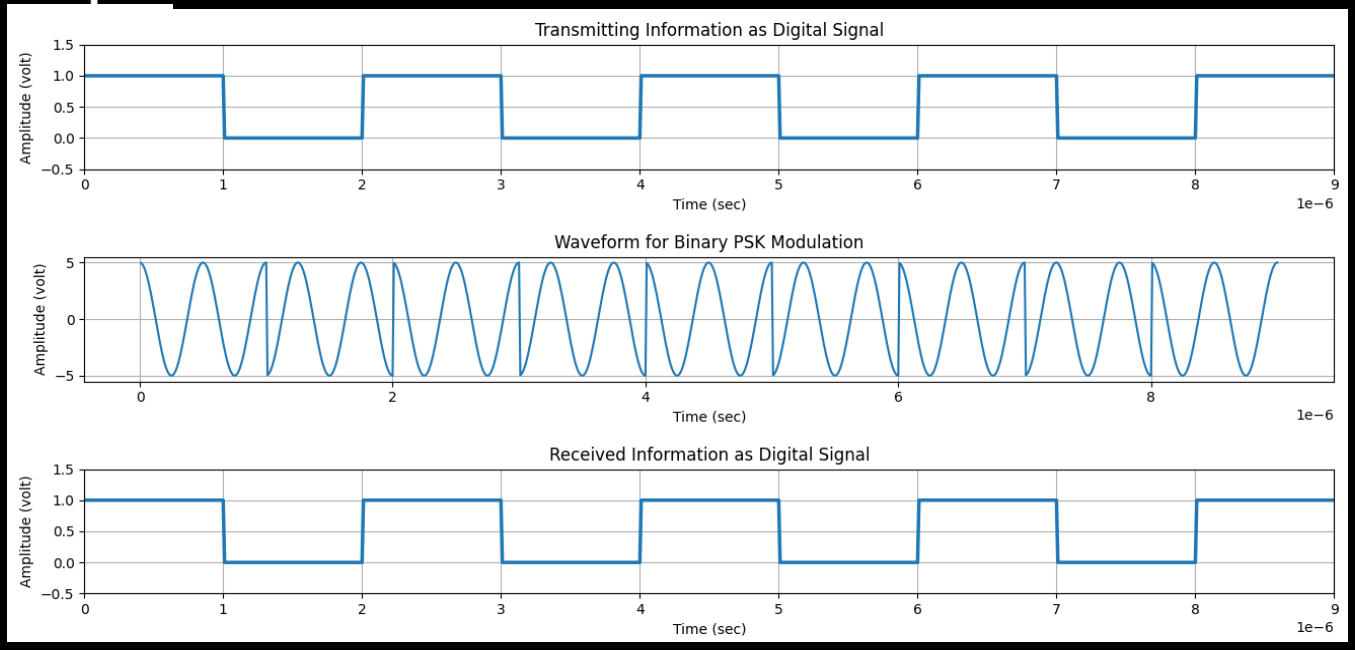
```
import numpy as np
import matplotlib.pyplot as plt
x = [1, 0, 1, 0, 1, 0, 1, 0, 1]
bp = 1e-6   # bit period
```

```
print("Binary Information at transmitter:", x)
bit = []
for n in range(len(x)):
    if x[n] == 1:
        se = np.ones(100)
    else:
        se = np.zeros(100)
    bit.extend(se)
bit = np.array(bit)
t1 = np.arange(bp/100, bp*len(x)+bp/100, bp/100)
plt.figure(figsize=(10,6))
plt.subplot(3,1,1)
plt.plot(t1, bit, linewidth=2.5)
plt.grid(True)
plt.axis([0, bp*len(x), -0.5, 1.5])
plt.ylabel('Amplitude (volt)')
plt.xlabel('Time (sec)')
plt.title('Transmitting Information as Digital Signal')
A = 5
br = 1 / bp
f = br * 2
t2 = np.arange(bp/100, bp + bp/100, bp/100)
ss = len(t2)
m = []
for i in range(len(x)):
    if x[i] == 1:
        y = A * np.cos(2 * np.pi * f * t2)
    else:
        y = A * np.cos(2 * np.pi * f * t2 + np.pi)
    m.extend(y)
m = np.array(m)
t3 = np.arange(bp/100, bp*len(x)+bp/100, bp/100)
```

```
plt.subplot(3,1,2)
plt.plot(t3, m)
plt.grid(True)
plt.xlabel('Time (sec)')
plt.ylabel('Amplitude (volt)')
plt.title('Waveform for Binary PSK Modulation')
mn = []
for n in range(ss, len(m)+1, ss):
    t = np.arange(bp/100, bp + bp/100, bp/100)
    y = A * np.cos(2 * np.pi * f * t)
    mm = y * m[n-ss:n]
    z = np.trapz(mm, t)
    zz = round(2 * z / bp)
    if zz > 0:
        a = 1
    else:
        a = 0
    mn.append(a)
print("Binary Information at Receiver After PSK
Demodulation:", mn)
bit_rec = []
for n in range(len(mn)):
    if mn[n] == 1:
        se = np.ones(100)
    else:
        se = np.zeros(100)
    bit_rec.extend(se)
bit_rec = np.array(bit_rec)
t4 = np.arange(bp/100, bp*len(mn)+bp/100, bp/100)
plt.subplot(3,1,3)
plt.plot(t4, bit_rec, linewidth=2.5)
plt.grid(True)
```

```
plt.axis([0, bp*len(mn), -0.5, 1.5])
plt.xlabel('Time (sec)')
plt.ylabel('Amplitude (volt)')
plt.title('Received Information as Digital Signal')
plt.tight_layout()
plt.show()
```

Output:



Result and Discussion:

The simulation successfully generates the Binary Phase Shift Keying (BPSK) waveform.

From the output:

- The first plot shows the original binary data sequence.
- The second plot represents the carrier signal.
- The third plot displays the PSK modulated signal.

It is observed that:

- When the input bit is ‘1’, the carrier signal remains unchanged.
- When the input bit is ‘0’, the carrier signal undergoes a phase shift of 180° (inverted waveform).
- The amplitude and frequency remain constant; only the phase changes.

Experiment No:04

Title: To create a program and simulate the Frequency Shift Keying (FSK) modulation technique.

Objective:

- To understand the basic concept of Frequency Shift Keying (FSK) modulation.
- To develop a Python program to generate and simulate an FSK signal.
- To analyze the waveform of the modulated signal.

Theory:

Frequency Shift Keying (FSK) is a digital modulation technique in which the frequency of the carrier signal is varied according to the digital input signal.

- In Binary FSK (BFSK), two different frequencies are used:
  - $f_1 \rightarrow$ represents binary **1**
  - $f_0 \rightarrow$ represents binary **0**
- The general form of FSK signal:
  - For bit 1:

$$s_1(t) = A\cos(2\pi f_1 t)$$

  - For bit 0:

$$s_0(t) = A\cos(2\pi f_0 t)$$

Where:

- A= Amplitude of the carrier signal
- f_1, f_0= Carrier frequencies
- t= Time

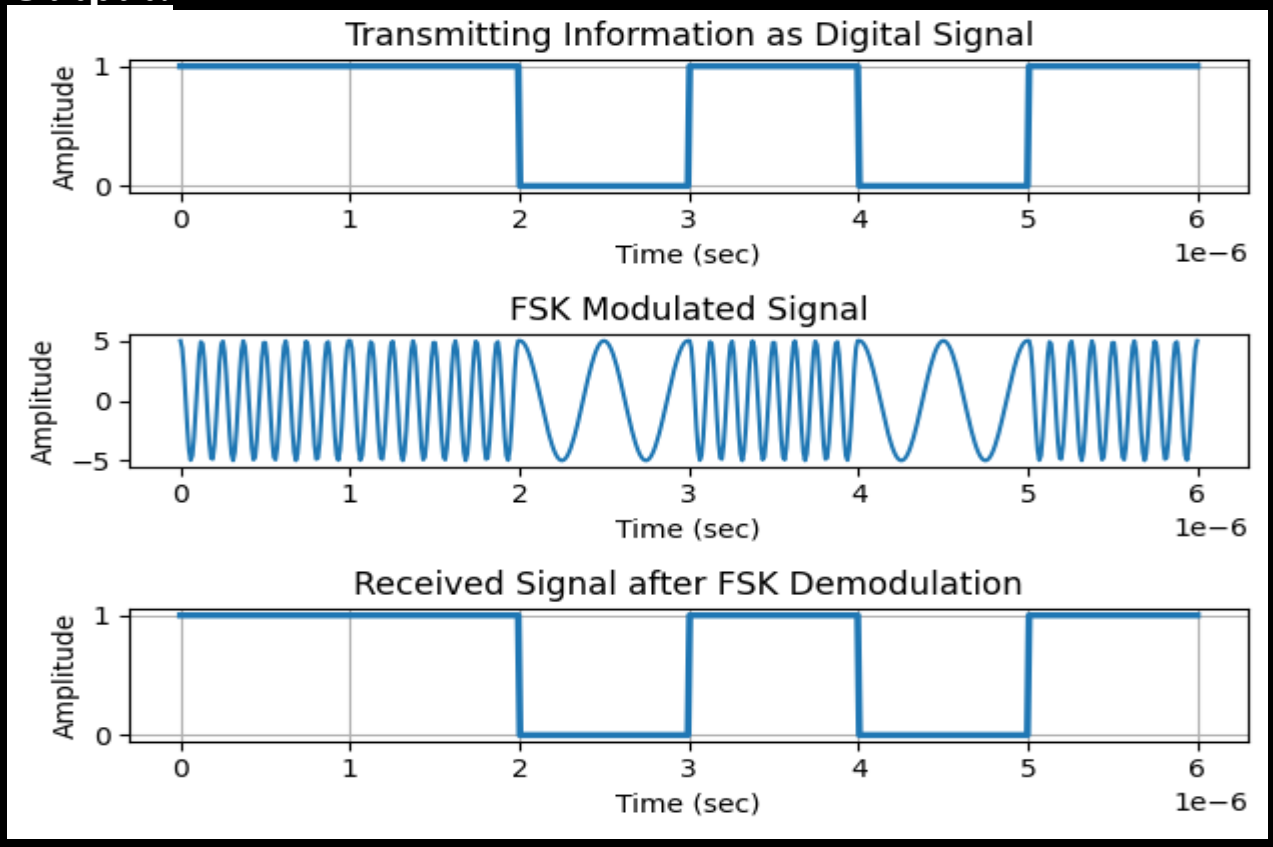
Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
x = [1, 1, 0, 1, 0, 1]
print("Binary information at transmitter:", x)
bp = 1e-6          # bit period
samples = 100      # samples per bit
```

```
bit = []
for b in x:
    if b == 1:
        bit.extend([1]*samples)
    else:
        bit.extend([0]*samples)
bit = np.array(bit)
t1 = np.linspace(0, bp*len(x), len(bit))
plt.subplot(3,1,1)
plt.plot(t1, bit, linewidth=2.5)
plt.title("Transmitting Information as Digital Signal")
plt.xlabel("Time (sec)")
plt.ylabel("Amplitude")
plt.grid()
A = 5
br = 1 / bp
f1 = br * 8     # frequency for bit 1
f2 = br * 2     # frequency for bit 0
t2 = np.linspace(0, bp, samples)
m = []
for b in x:
    if b == 1:
        y = A * np.cos(2 * np.pi * f1 * t2)
    else:
        y = A * np.cos(2 * np.pi * f2 * t2)
    m.extend(y)
m = np.array(m)
t3 = np.linspace(0, bp*len(x), len(m))
plt.subplot(3,1,2)
plt.plot(t3, m)
plt.title("FSK Modulated Signal")
plt.xlabel("Time (sec)")
```

```
plt.ylabel("Amplitude")
mn = []
ss = samples
for i in range(0, len(m), ss):
    segment = m[i:i+ss]
    t = np.linspace(0, bp, samples)
    y1 = np.cos(2 * np.pi * f1 * t)
    y2 = np.cos(2 * np.pi * f2 * t)
    z1 = np.trapz(segment * y1, t)
    z2 = np.trapz(segment * y2, t)
    if z1 > z2:
        mn.append(1)
    else:
        mn.append(0)
print("Binary information at receiver:", mn)
bit_rec = []
for b in mn:
    if b == 1:
        bit_rec.extend([1]*samples)
    else:
        bit_rec.extend([0]*samples)
bit_rec = np.array(bit_rec)
t4 = np.linspace(0, bp*len(mn), len(bit_rec))
plt.subplot(3,1,3)
plt.plot(t4, bit_rec, linewidth=2.5)
plt.title("Received Signal after FSK Demodulation")
plt.xlabel("Time (sec)")
plt.ylabel("Amplitude")
plt.grid()
plt.tight_layout()
plt.show()
```

Output:



Result and Discussion:

The simulation successfully generates the FSK modulated signal based on the given binary input sequence.

- The upper plot represents the input binary data.
- The lower plot shows the FSK waveform.

From the output:

- When the input bit is **1**, the signal oscillates with a **higher frequency (f1)**.
- When the input bit is **0**, the signal oscillates with a **lower frequency (f0)**.
- The amplitude remains constant throughout the signal.

This confirms that:

- FSK modulation changes **frequency** based on input data.
- It is robust against noise compared to amplitude-based modulation techniques.

Experiment No:-05

Title: To implement a program and simulate the Unipolar Non-Return-to-Zero (NRZ) signaling technique.

Objective:

- To implement a program that generates a Unipolar NRZ signal.
- To simulate and visualize the waveform of Unipolar NRZ signaling.
- To understand how binary data is represented using Unipolar NRZ technique.

Theory:

Unipolar Non-Return-to-Zero (NRZ) is a basic digital line coding technique used in data communication.

Types of NRZ:

- NRZ-Level (NRZ-L)
- NRZ-Inverted (NRZ-I)

In this experiment, we use **Unipolar NRZ-L**.

Advantages:

- Simple to implement
- Efficient bandwidth usage

Disadvantages:

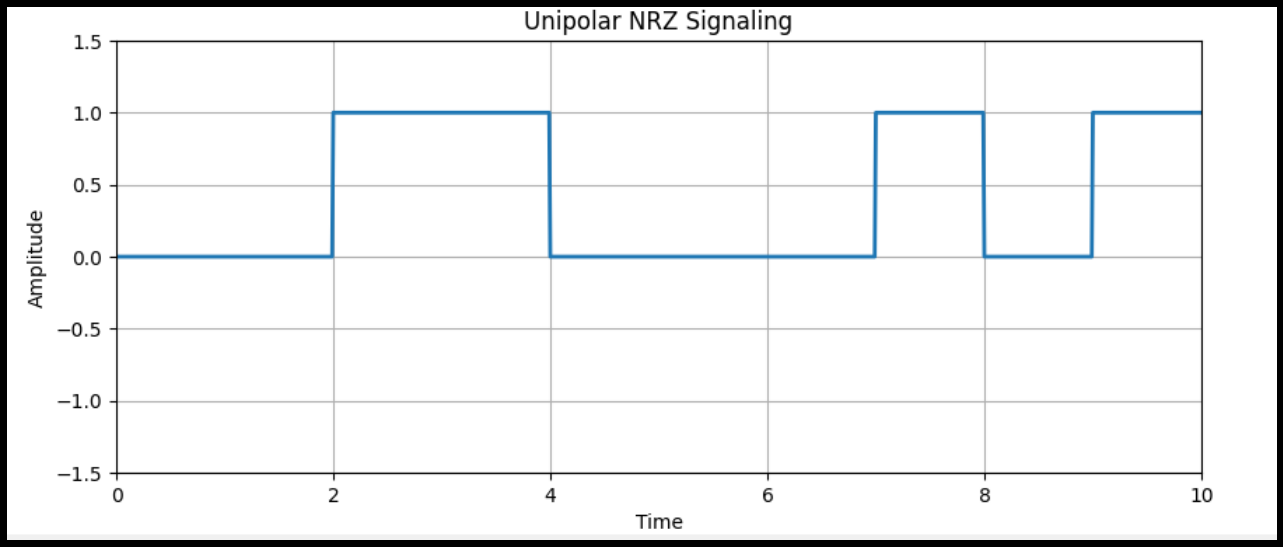
- Synchronization problem (long sequence of 0s or 1s)
- DC component present

Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
import random
N = 10  # Number of bits
n = [random.randint(0, 1) for _ in range(N)] # Random bit generation
print(f"Generated bits (n): {n}")
nn = []
```

```
for m in range(N):
    if n[m] == 1:
        nn.append(1)
    else:
        nn.append(0)
print(f"Mapped values (nn): {nn}")
t = np.arange(0, N, 0.01) # Time duration set up
y = np.zeros(len(t))
for j in range(len(t)):
    i = int(np.floor(t[j]))
    if i < N:
        y[j] = nn[i]
plt.figure(figsize=(10, 4))
plt.plot(t, y, linewidth=2)
plt.axis([0, N, -1.5, 1.5]) # Axis set-up
plt.grid(True)
plt.title("Unipolar NRZ Signaling")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.show()
```

Output:



Result and Discussion:

Result:

- The program successfully generates a Unipolar NRZ waveform.
- Binary **1 is represented by a high level (1V)** and binary **0 by 0V**.
- The signal remains constant during each bit duration without returning to zero.

Discussion:

- From the waveform, it is observed that consecutive **1s produce a continuous high signal**, and consecutive **0s produce a flat zero signal**.
- There is **no transition between identical bits**, which can create synchronization issues at the receiver.
- The presence of a **DC component** makes it less suitable for long-distance communication.
- However, due to its simplicity, Unipolar NRZ is useful for understanding basic digital communication concepts.

Experient No:-06

Title: To model a program and simulate the Polar Non-Return-to-Zero (NRZ) signaling technique.

Objective:

- To develop a program to simulate Polar NRZ signaling.
- To analyze the waveform representation of binary data using Polar NRZ technique.

Theory:

Polar NRZ (Non-Return-to-Zero) is a type of line coding technique used in digital communication systems. In this method:

- Binary **1** is represented by a **positive voltage level (+V)**
- Binary **0** is represented by a **negative voltage level (−V)**

Types of Polar NRZ:

- **NRZ-L (Level):** Level depends on bit value
- **NRZ-I (Inverted):** Transition occurs on bit 1

In this experiment, we use **Polar NRZ-L**, where:

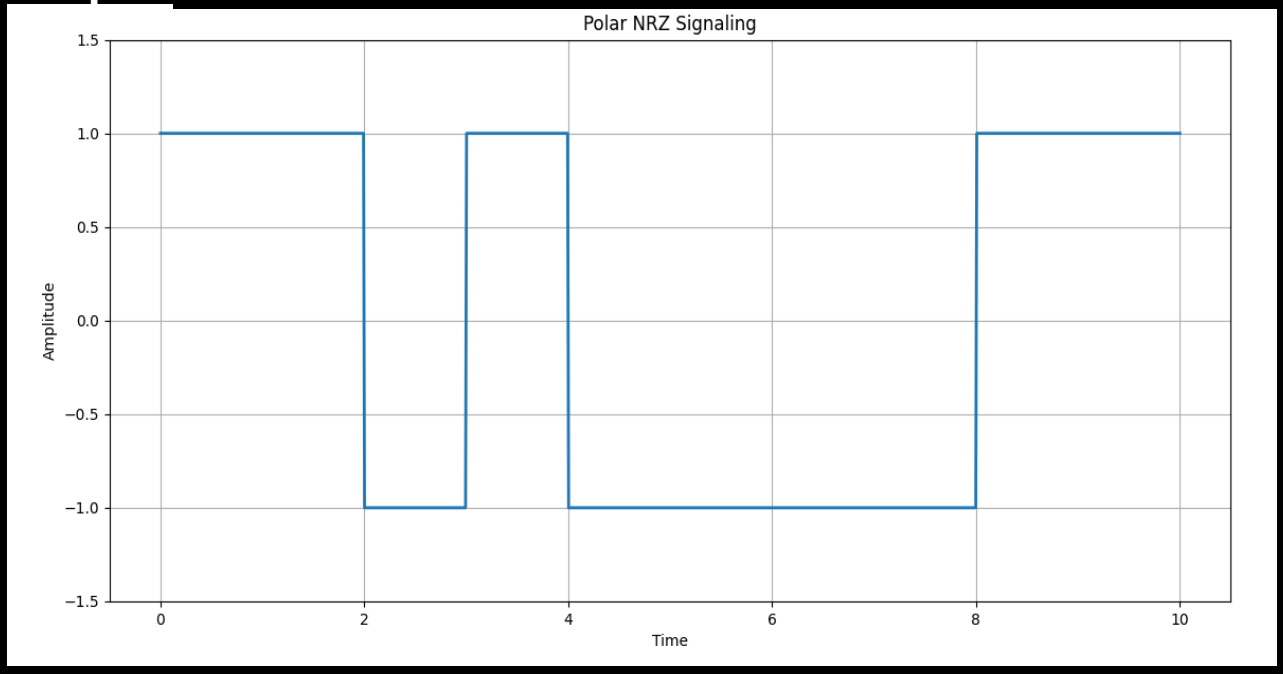
$$1 \rightarrow +A, 0 \rightarrow -A$$

Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
N = 10
n = np.random.randint(0, 2, N)
print("Random bits:", n)
nn = np.where(n == 1, 1, -1)
print("Mapped bits (+1/-1):", nn)
samples_per_bit = 100  # Number of points per bit
y = np.repeat(nn, samples_per_bit)  # Repeat each value for
plotting
t = np.linspace(0, N, N * samples_per_bit)  # Time axis
plt.figure(figsize=(10,3))
plt.plot(t, y, linewidth=2)
plt.title("Polar NRZ Signaling")
```

```
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.ylim([-1.5, 1.5])
plt.show()
```

Output:



Result and Discussion:

The simulated waveform clearly represents the Polar NRZ signaling technique:

- Binary **1** is shown as a **positive amplitude (+1)**
- Binary **0** is shown as a **negative amplitude (−1)**
- The signal remains constant during each bit interval and does not return to zero between bits

Observations:

- The waveform has **sharp transitions** only when the bit value changes.
- Long sequences of 1s or 0s produce **continuous flat levels**, which may cause synchronization issues.
- The method is **bandwidth efficient** but lacks inherent clocking information.

Experiment No:-07

Title: To write a program and simulate the Unipolar Return-to-Zero (RZ) signaling technique.

Objective:

- To develop a Python program for generating a Unipolar Return-to-Zero (RZ) signal.
- To simulate and analyze the waveform of the RZ signaling technique.

Theory:

Unipolar Return-to-Zero (RZ) is a type of line coding technique used in digital communication systems. In this method:

- Binary **1** is represented by a positive voltage for half of the bit duration and then returns to zero.
- Binary **0** is represented by zero voltage for the entire bit duration.

Waveform Behavior:

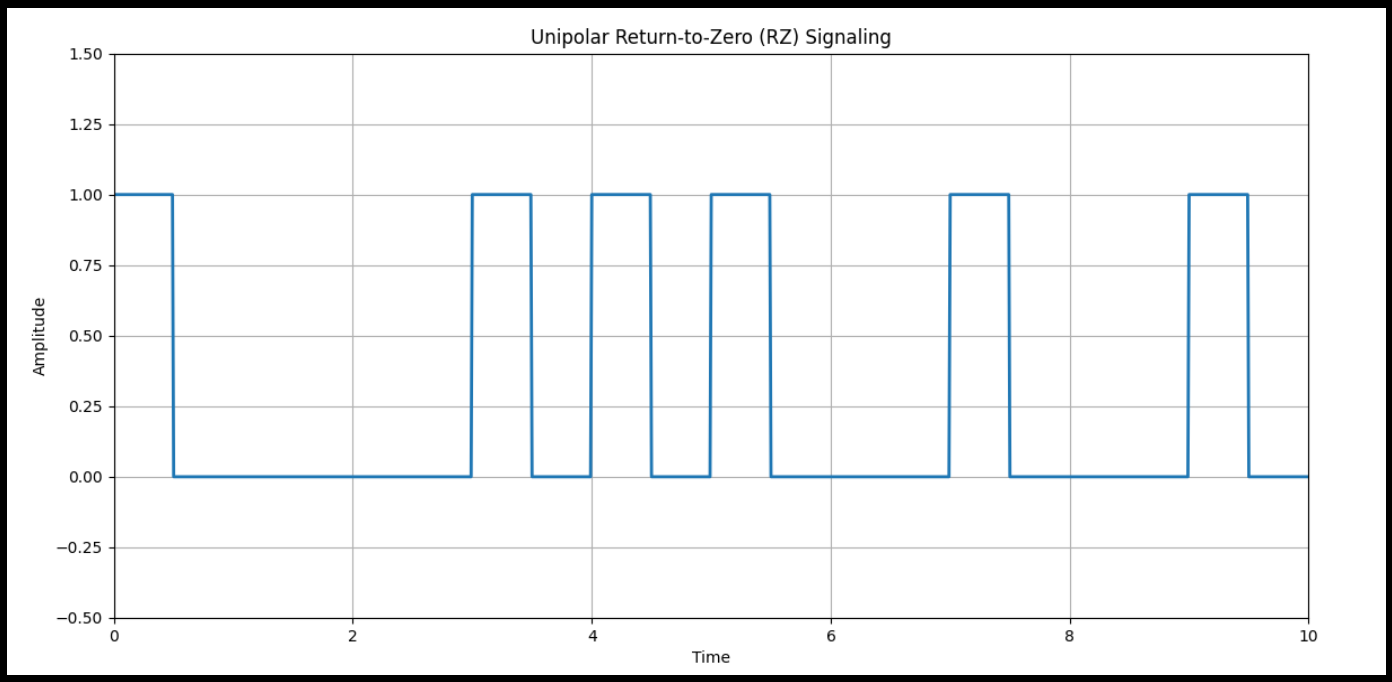
- For bit **1** → **High (first half)** → **Zero (second half)**
- For bit **0** → **Zero (entire duration)**

Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
N = 10
n = np.random.randint(0, 2, N)
print("Input bits:", n)
y = []
t = np.arange(0, N, 0.01)  # time vector
dt = t[1] - t[0]
i = 0
for j in range(len(t)):
    if i < N:
        if i <= t[j] < i + 0.5:
            y.append(n[i])  # first half-cycle
```

```
        elif i + 0.5 <= t[j] < i + 1:
            y.append(0)     # second half-cycle
        else:
            i += 1
            # check again for new bit
            if i < N:
                if i <= t[j] < i + 0.5:
                    y.append(n[i])
                else:
                    y.append(0)
            else:
                y.append(0)
    else:
        y.append(0)
plt.figure(figsize=(10, 3))
plt.plot(t, y, linewidth=2)
plt.title('Unipolar Return-to-Zero (RZ) Signaling')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.grid(True)
plt.axis([0, N, -0.5, 1.5])
plt.show()
```

Output:



Result and Discussion:

The simulated waveform of the Unipolar RZ signal clearly shows the behavior of the encoding scheme:

- For each binary **1**, the signal rises to a positive level during the first half of the bit period and then returns to zero in the second half.
- For binary **0**, the signal remains at zero throughout the entire bit duration.
- The waveform contains frequent transitions, which help in maintaining synchronization between transmitter and receiver.
- Compared to NRZ, the RZ signal occupies more bandwidth because each bit introduces at least one transition.
- The presence of zero levels between pulses reduces the chances of baseline wandering.

Experiment No:-08

Title: To simulate a program for the Bipolar Return-to-Zero (RZ) signaling technique.

Objective:

- To develop a Python program to simulate Bipolar Return-to-Zero (RZ) signaling.
- To analyze the waveform characteristics of Bipolar RZ signals.

Theory:

Bipolar Return-to-Zero (RZ) is a type of line coding technique used in digital communication systems. In this method:

- Logical **1** is represented by alternating positive and negative voltages (+V, -V).
- Logical **0** is represented by zero voltage (0V).
- The signal **returns to zero** in the middle of each bit period.

Working Principle:

- For every bit:
  - If bit = 1 → output is either +V or -V (alternating), then returns to 0
  - If bit = 0 → output remains 0
- Each bit duration is divided into two halves:
  - First half → signal value
  - Second half → zero

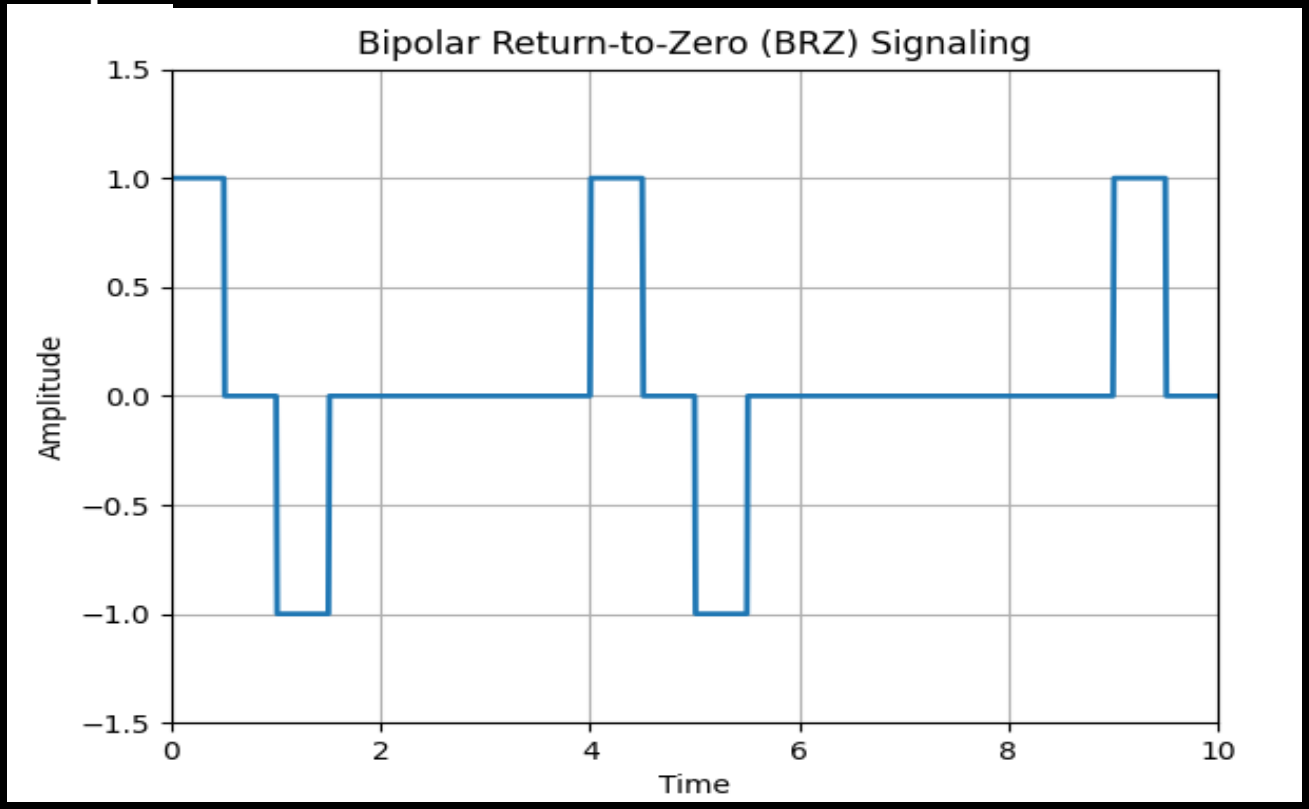
Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
N = 10
n = np.random.randint(0, 2, N)
print("Random bits:", n)
f = 1
nn = np.zeros(N)
for m in range(N):
```

```
    if n[m] == 1:
        if f == 1:
            nn[m] = 1
            f = -1
        else:
            nn[m] = -1
            f = 1
    else:
        nn[m] = 0
print("Bipolar sequence:", nn)
i = 0        # Python index starts from 0
a = 0        # Initial value for the first half cycle
b = 0.5      # Initial value for the second half cycle
t = np.arange(0, N, 0.01)
y = np.zeros(len(t))
for j in range(len(t)):
    if t[j] >= a and t[j] <= b:
        y[j] = nn[i]          # First half cycle
    elif t[j] > b and t[j] <= i + 1:
        y[j] = 0              # Second half cycle
    else:
        i += 1
        a += 1
        b += 1
        if i < N:
            y[j] = nn[i]
        else:
            y[j] = 0
plt.plot(t, y, linewidth=2)
plt.axis([0, N, -1.5, 1.5])
plt.grid(True)
plt.title('Bipolar Return-to-Zero (BRZ) Signaling')
```

```
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.show()
```

Output:



Result and Discussion:

The simulation successfully generates the Bipolar Return-to-Zero (RZ) waveform for the given binary sequence. The output graph shows:

- Logical **1s** are represented by alternating positive and negative pulses.
- Logical **0s** remain at zero level throughout the bit duration.
- Each pulse returns to zero in the middle of the bit period, confirming the RZ property.

Observations:

- The alternating polarity of 1s helps eliminate the DC component.
- Frequent transitions improve clock synchronization at the receiver.

Experiment No:-09

Title: To develop a program and simulate the Manchester (Split-Phase) coding technique.

Objective:

- To develop a program for generating Manchester encoded signals.
- To simulate and analyze the waveform of Manchester (split-phase) coding.
- To understand the transition behavior of digital data in Manchester encoding.

Theory:

Manchester encoding is a digital line coding technique used in data communication where each bit has a transition in the middle of the bit period.

Encoding Rules:

- Binary **1** → High to Low transition
- Binary **0** → Low to High transition

Advantages:

- No DC component
- Easy clock recovery
- Error detection possible

Disadvantages:

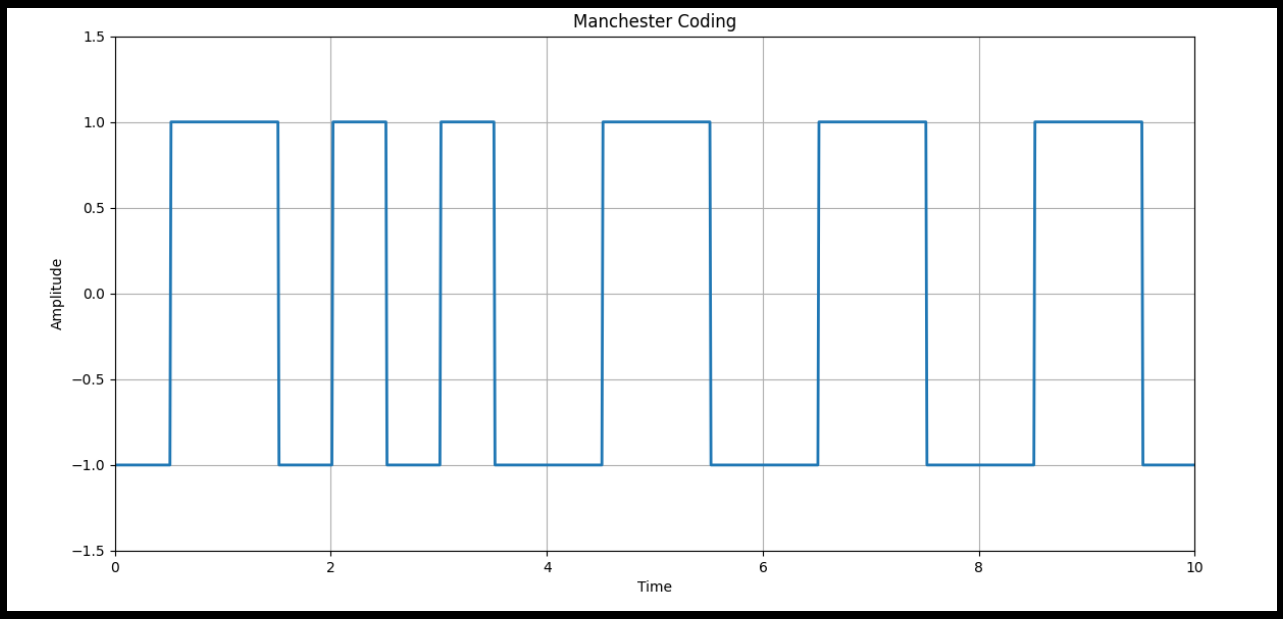
- Requires double bandwidth compared to NRZ

Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
N = 10
n = np.random.randint(0, 2, N)
print("Random bits:", n)
nnn = []
for bit in n:
    if bit == 1:
```

```
        nn = [1, -1]
    else:
        nn = [-1, 1]
    nnn.extend(nn)
print("Manchester sequence:", nnn)
i = 0
l = 0.5
t = np.arange(0, N, 0.01)
y = np.zeros(len(t))
for j in range(len(t)):
    if t[j] <= l:
        y[j] = nnn[i]
    else:
        y[j] = nnn[i]
        i += 1
        l += 0.5
plt.figure(figsize=(10, 3))
plt.plot(t, y, linewidth=2)
plt.axis([0, N, -1.5, 1.5])
plt.grid(True)
plt.title("Manchester Coding")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.show()
```

Output:



Result and Discussion:

Result:

- The program successfully generates the Manchester encoded waveform for the given binary data 1011001.
- Each bit is represented by two signal levels showing a transition in the middle.
- The plotted waveform clearly shows:
  - High-to-Low transitions for binary 1
  - Low-to-High transitions for binary 0

Discussion:

- The simulation verifies that Manchester encoding ensures a transition in every bit period, making synchronization reliable.
- Compared to NRZ encoding, this method increases signal transitions, which improves clock recovery but doubles bandwidth usage.
- The waveform shows uniform timing and clear transitions, which helps reduce errors in communication systems.

Experiment No:-10

Title: Perform the QPSK modulation and demodulation using MATLAB/ Python. Display the signal and its constellation.

Objective:

- To perform QPSK modulation and demodulation.
- To visualize the modulated signal waveform.
- To plot and analyze the constellation diagram.
- To recover the original binary data from the received signal.

Theory:

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique where two bits are transmitted per symbol by changing the phase of the carrier signal.

- Each symbol represents **2 bits**
- Total phases used: **4 (0°, 90°, 180°, 270°)**

Bit to Phase Mapping:

Bits	Phase
00	0°
01	90°
10	180°
11	270°

QPSK signal can be expressed as:

$$s(t) = I\cos(2\pi f t) + Q\sin(2\pi f t)$$

Where:

- I= In-phase component
- Q= Quadrature component

Constellation Diagram

- Represents symbols in the I-Q plane
- Four points corresponding to four phases

Demodulation

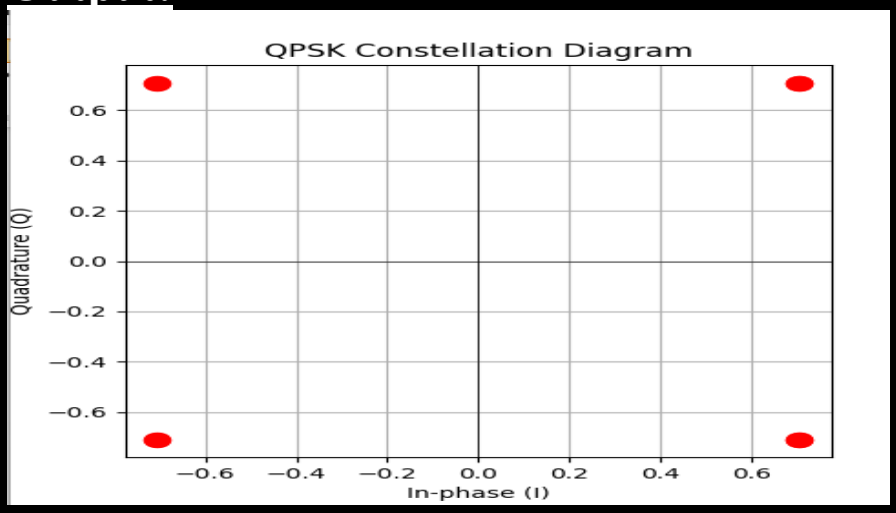
- Received signal is compared with reference signals
- Closest constellation point determines transmitted bits

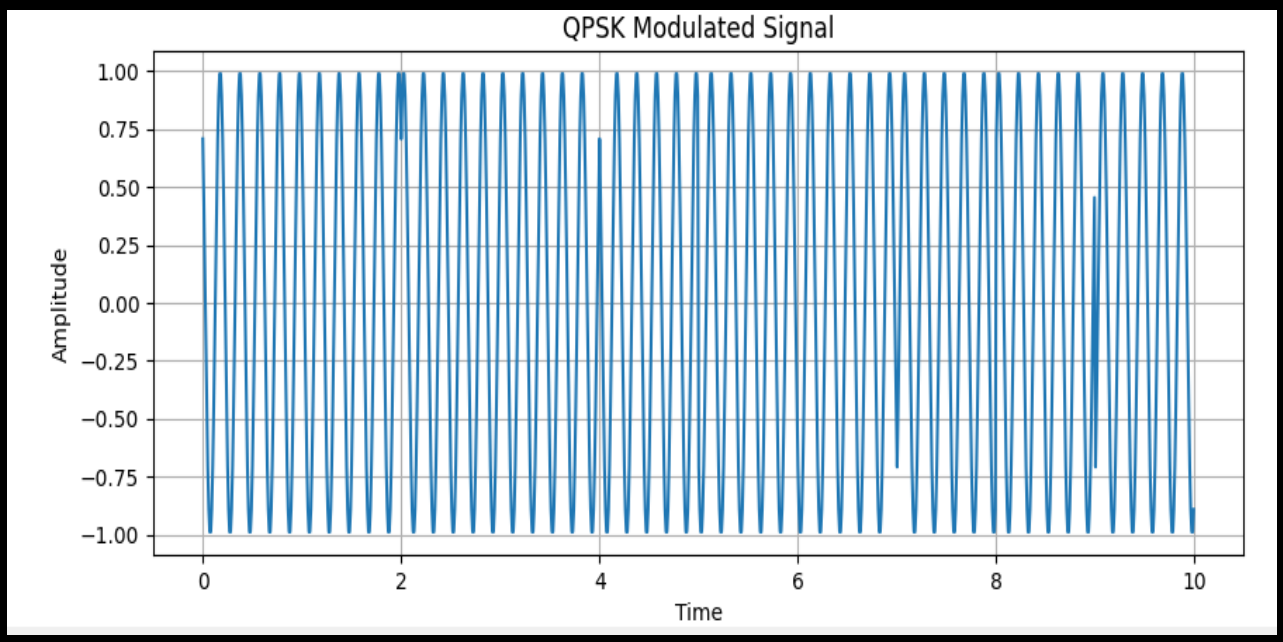
Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
num_bits = 20                    # Number of bits
fc = 5                           # Carrier frequency (arbitrary for plotting)
fs = 100                         # Sampling frequency
Tb = 1                           # Bit duration
t_bit = np.arange(0, Tb, 1/fs)    # Time vector for one bit
data = np.random.randint(0, 2, num_bits)
print("Original Bits: ", data)
if len(data) % 2 != 0:           # Make even if odd number of bits
    data = np.append(data, 0)
bit_pairs = data.reshape(-1, 2)
mapping = {
    (0,0): (1/np.sqrt(2), 1/np.sqrt(2)),
    (0,1): (-1/np.sqrt(2), 1/np.sqrt(2)),
    (1,1): (-1/np.sqrt(2), -1/np.sqrt(2)),
    (1,0): (1/np.sqrt(2), -1/np.sqrt(2))
}
symbols = np.array([mapping[tuple(b)] for b in bit_pairs])
I = symbols[:,0]
Q = symbols[:,1]
qpsk_signal = np.array([])
for i_val, q_val in zip(I, Q):
    symbol_wave = i_val * np.cos(2*np.pi*fc*t_bit) - q_val *
np.sin(2*np.pi*fc*t_bit)
    qpsk_signal = np.concatenate((qpsk_signal, symbol_wave))
t = np.arange(0, len(qpsk_signal)) / fs
plt.figure(figsize=(10,4))
plt.plot(t, qpsk_signal)
plt.title("QPSK Modulated Signal")
plt.xlabel("Time")
```

```
plt.ylabel("Amplitude")
plt.grid()
plt.figure(figsize=(5,5))
plt.scatter(I, Q, color='r', s=100)
plt.title("QPSK Constellation Diagram")
plt.xlabel("In-phase (I)")
plt.ylabel("Quadrature (Q)")
plt.axhline(0, color='k', linewidth=0.5)
plt.axvline(0, color='k', linewidth=0.5)
plt.grid()
demod_bits = []
for i_val, q_val in zip(I, Q):
    # decision thresholds at 0
    bit1 = 0 if i_val > 0 else 1
    bit2 = 0 if q_val > 0 else 1
    demod_bits.extend([bit1, bit2])
demod_bits = np.array(demod_bits)
print("Demodulated Bits:", demod_bits)
plt.show()
```

Output:





Result and Discussion:

The QPSK modulation and demodulation process was successfully implemented using Python. The generated waveform shows how the signal changes according to different symbol phases. The constellation diagram clearly displays four distinct points representing the four phase states of QPSK.

From the output, it is observed that the demodulated bits match the original transmitted bits, indicating error-free transmission under ideal conditions (no noise). This confirms the correctness of the modulation and demodulation process.

Experiment No:-11

Title: Generate 16-QAM modulation and obtain the QAM constellation using MATLAB/ Python.

Objective:

- To generate a **16-QAM (Quadrature Amplitude Modulation)** signal.
- To visualize and analyze the **constellation diagram** of 16-QAM.
- To understand how digital data is mapped into amplitude and phase variations.

Theory:

Quadrature Amplitude Modulation (QAM) is a digital modulation technique that combines both **amplitude modulation** and **phase modulation**.

In **16-QAM**,

- Total symbols = 16
- Each symbol represents **4 bits** (since $2^4 = 16$)
- The signal is formed using two carriers:
  - **In-phase (I) component**
  - **Quadrature (Q) component**

The transmitted signal can be written as:

$$s(t) = I(t)\cos(2\pi f_c t) + Q(t)\sin(2\pi f_c t)$$

Where:

- $I(t)$= In-phase component
- $Q(t)$= Quadrature component
- f_c= Carrier frequency

In 16-QAM:

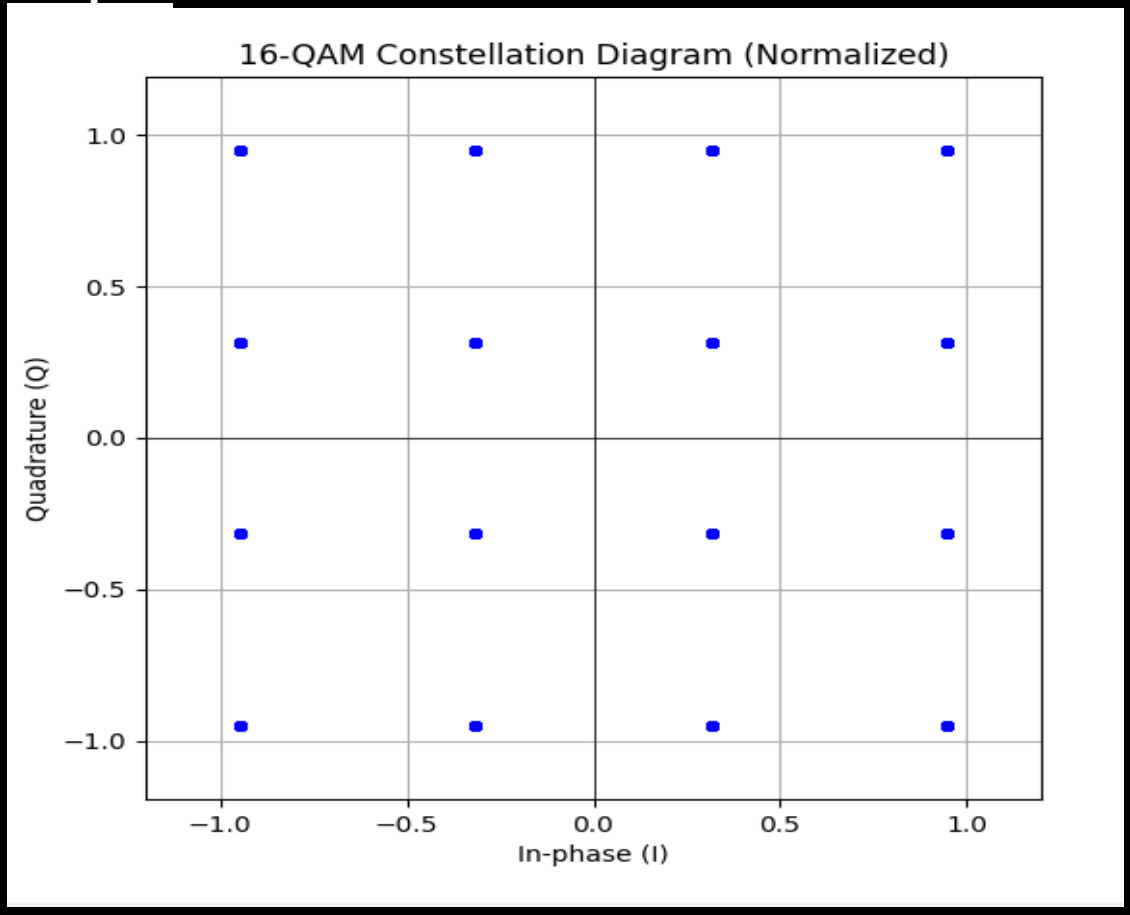
- I and Q take values from: **{-3, -1, +1, +3}**
- These create a **4×4 grid constellation**

Each point in the constellation represents a unique 4-bit combination.

Source Code :

```
import numpy as np
import matplotlib.pyplot as plt
N = 1000
data = np.random.randint(0, 16, N)
I = 2*(data % 4) - 3
Q = 2*(data // 4) - 3
I = I / np.sqrt(10)
Q = Q / np.sqrt(10)
plt.figure(figsize=(6,6))
plt.scatter(I, Q, color='blue', s=15, marker='o')  # clearer markers
plt.title("16-QAM Constellation Diagram (Normalized)")
plt.xlabel("In-phase (I)")
plt.ylabel("Quadrature (Q)")
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.axis('equal')  # ensures x and y scales are equal
plt.xlim(-1.2, 1.2)
plt.ylim(-1.2, 1.2)
plt.show()
```

Output:



Result and Discussion:

The 16-QAM modulation was successfully generated using Python, and the corresponding constellation diagram was obtained. The output shows 16 distinct symbol points arranged symmetrically in a square grid, confirming correct mapping of data symbols into I and Q components.

Each constellation point represents 4 bits of information, making 16-QAM more bandwidth-efficient compared to lower-order modulation schemes like QPSK.

From the diagram, it is clear that increasing the number of symbols increases data rate but reduces noise immunity. Therefore, 16-QAM provides a trade-off between **data rate and reliability**, making it widely used in modern communication systems such as Wi-Fi and LTE.

Experiment No:-12

Title: Perform sampling and reconstruction of continuous signal using MATLAB/ Python.

Objective:

1. To understand the concept of sampling a continuous-time signal.
2. To observe the effect of sampling frequency on signal reconstruction.
3. To reconstruct the original continuous-time signal from its samples.

Theory:

Sampling is the process of converting a continuous-time signal $x(t)$into a discrete-time signal $x[n]$by taking values at uniform intervals T_s(sampling period).

- **Sampling Theorem (Nyquist Theorem):**
  A continuous signal can be perfectly reconstructed from its samples if the sampling frequency $f_s$is greater than twice the maximum frequency $f_m$present in the signal:

$$f_s > 2f_m$$

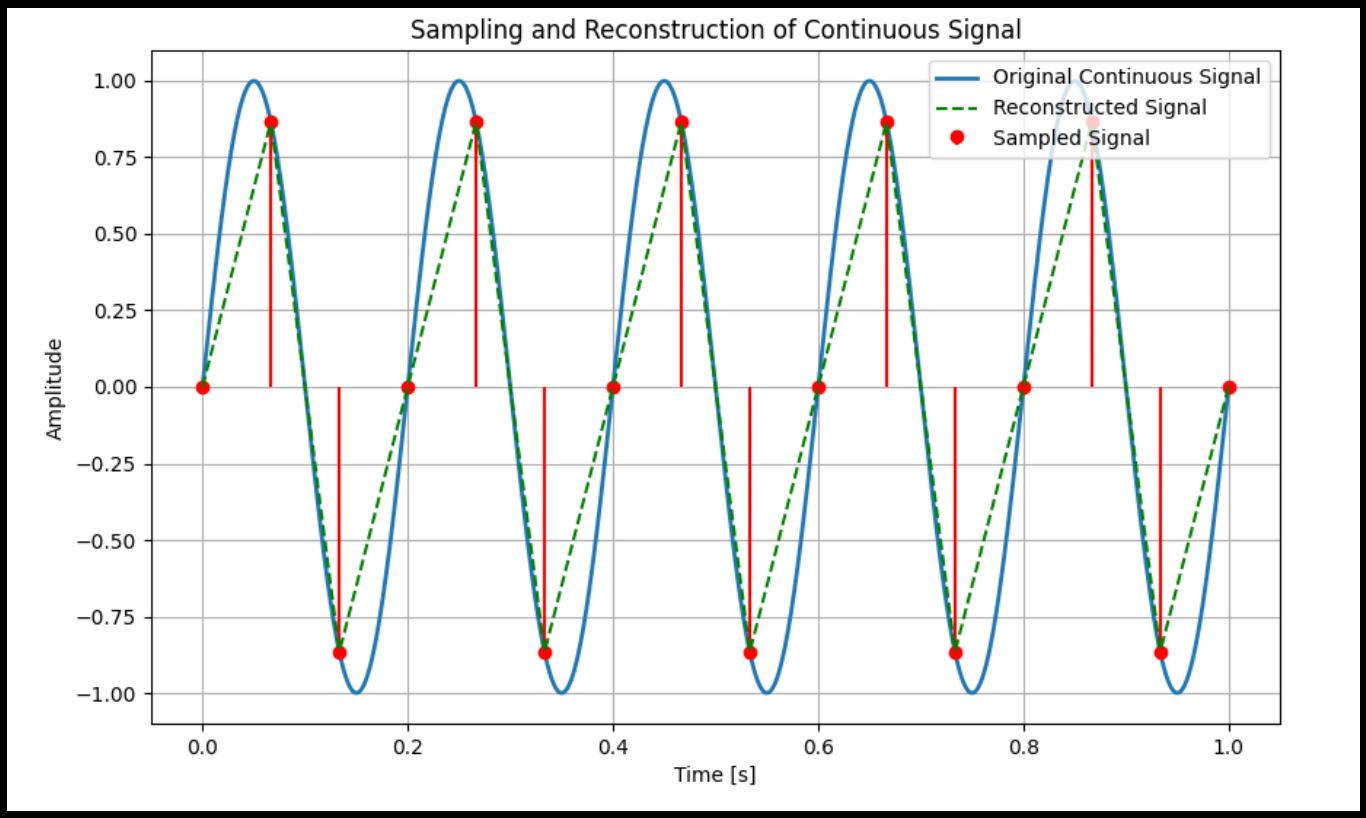
where $f_s = \frac{1}{T_s}$.

- **Reconstruction:**
  The original signal can be reconstructed from its samples using interpolation, usually ideal low-pass filtering.
- **Effect of Undersampling:**
  If $f_s < 2f_m$, aliasing occurs, and the signal cannot be perfectly reconstructed.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
f = 5            # Signal frequency in Hz
fs = 15          # Sampling frequency in Hz (fs > 2*f to satisfy
Nyquist)
t_end = 1        # Duration of signal in seconds
t = np.linspace(0, t_end, 1000)  # dense time vector
x_cont = np.sin(2 * np.pi * f * t)
Ts = 1 / fs
t_samples = np.arange(0, t_end + Ts, Ts)  # include last point
x_samples = np.sin(2 * np.pi * f * t_samples)
reconstruct = interp1d(t_samples, x_samples, kind='linear')
x_reconstruct = reconstruct(t)
plt.figure(figsize=(10,6))
plt.plot(t, x_cont, label='Original Continuous Signal',
linewidth=2)
plt.stem(t_samples, x_samples, linefmt='r', markerfmt='ro',
basefmt=" ", label='Sampled Signal')
plt.plot(t, x_reconstruct, 'g--', label='Reconstructed Signal')
plt.title('Sampling and Reconstruction of Continuous Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.show()
```

Output:



Result and Discussion:

1. When the sampling frequency f_s is greater than twice the signal frequency f (e.g., $f_s = 15\text{Hz}$ for $f = 5\text{Hz}$), the reconstructed signal closely follows the original continuous signal. This demonstrates **Nyquist theorem**.
2. If f_s is less than twice the signal frequency (undersampling), the reconstructed signal deviates from the original, and **aliasing** occurs, leading to incorrect signal reconstruction.
3. Linear interpolation reconstructs the signal reasonably, but ideal reconstruction uses a **low-pass filter** for smoother approximation.
4. The lab demonstrates the importance of selecting an appropriate sampling frequency for accurate digital representation and reconstruction of analog signals.

Experiment No:-13

Title: To generate the waveform of Quadrature Phase Shift Keying signal using MATLAB/ Python.

Objective:

- To understand the concept of Quadrature Phase Shift Keying (QPSK).
- To generate a QPSK signal waveform using Python.
- To analyze the characteristics of the QPSK waveform.

Theory:

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique where two bits of data are represented by one of four possible phase shifts (0°, 90°, 180°, 270°) of a carrier signal. It is widely used in communication systems because it doubles the data rate compared to Binary Phase Shift Keying (BPSK) without increasing bandwidth.

The general form of a QPSK signal is:

$$s(t) = I(t)\cos(2\pi f_c t) + Q(t)\sin(2\pi f_c t)$$

Where:

- $I(t)$is the in-phase component (represents first bit).
- $Q(t)$is the quadrature component (represents second bit).
- $f_c$is the carrier frequency.

The bit pairs $(00, 01, 10, 11)$are mapped to phase angles $(45°, 135°, 225°, 315°)$or equivalently to $(\pi/4, 3\pi/4, 5\pi/4, 7\pi/4)$.

QPSK is more bandwidth-efficient than BPSK and is robust for noisy channels, making it ideal for modern wireless communications.

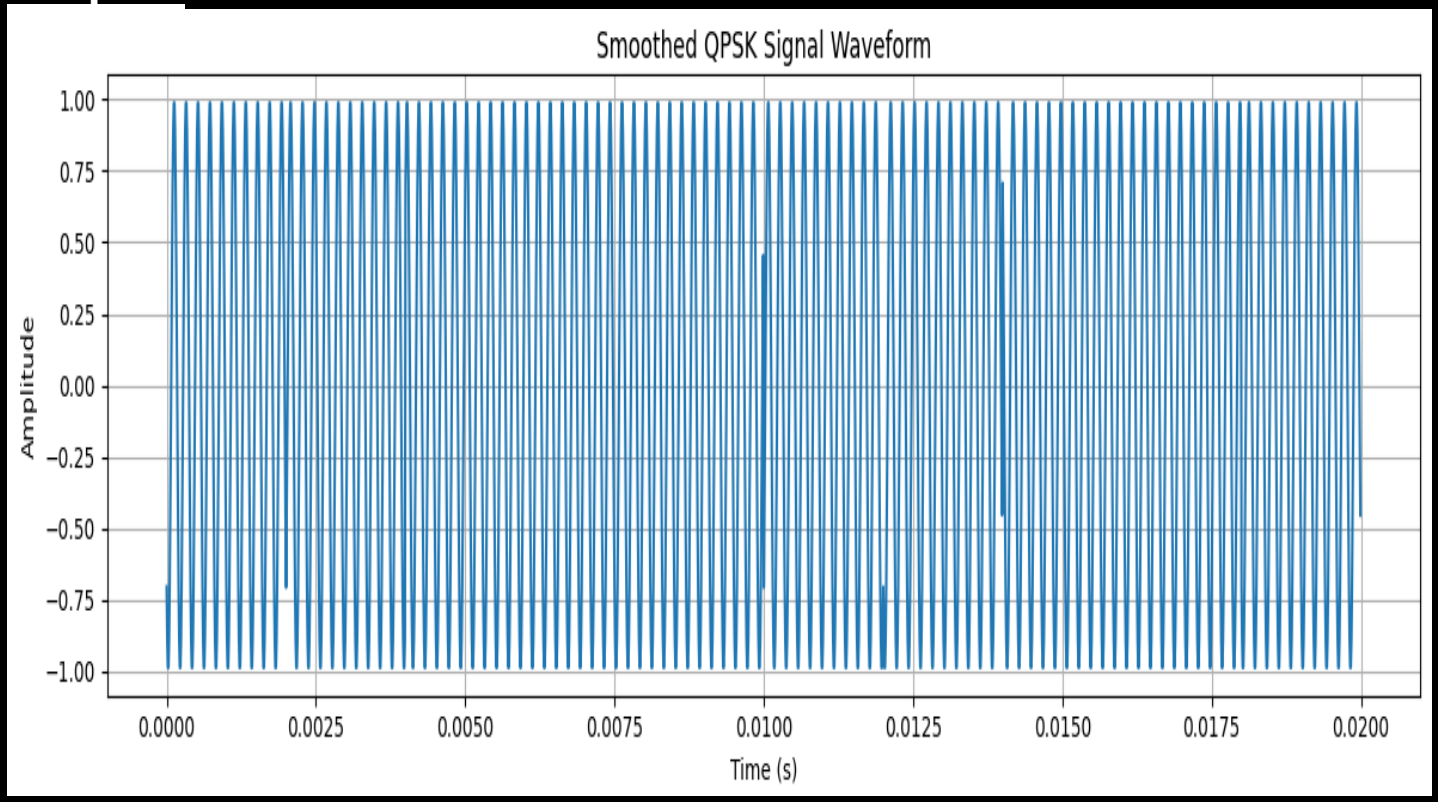
Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
bit_rate = 1e3        # Bit rate (bits per second)
fc = 5e3              # Carrier frequency (Hz)
Tb = 1/bit_rate       # Bit duration per bit
num_bits = 20         # Total number of bits
bits_per_symbol = 2   # QPSK uses 2 bits per symbol
num_symbols = num_bits // bits_per_symbol
N = 200
np.random.seed(0)
data_bits = np.random.randint(0, 2, num_bits)
bit_pairs = data_bits.reshape((-1, bits_per_symbol))
symbol_map = {
    (0, 0): np.pi/4,
    (0, 1): 3*np.pi/4,
    (1, 0): 5*np.pi/4,
    (1, 1): 7*np.pi/4
}
phases = np.array([symbol_map[tuple(b)] for b in bit_pairs])
t = np.linspace(0, Tb*num_bits, N*num_symbols,
endpoint=False)
qpsk_signal = np.zeros(len(t))
for i, ph in enumerate(phases):
    start_idx = i * N
    end_idx = (i + 1) * N
    # Generate carrier for this symbol
    qpsk_signal[start_idx:end_idx] = np.cos(2 * np.pi * fc *
t[start_idx:end_idx] + ph)
plt.figure(figsize=(12, 4))
plt.plot(t, qpsk_signal, linewidth=1.5)
plt.title("Smoothed QPSK Signal Waveform")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
print("Data Bits (in pairs):")
print(bit_pairs)
print("Mapped Phases (radians):")
print(phases)
```

Output :



Result and discussion:

1. QPSK successfully doubles the data rate compared to BPSK because each symbol carries 2 bits.
2. The waveform transitions show abrupt phase changes at symbol boundaries.
3. The generated signal can be transmitted over a communication channel and demodulated using coherent detection to recover the original bit stream.
4. The use of numpy arrays and reshaping makes mapping bits to symbols very efficient for simulations.

Experiment No:-14

Title: Perform the PAM modulation and demodulation using MATLAB/ Python.

Objective:

To understand and simulate the process of Pulse Amplitude Modulation (PAM) and its demodulation using Python, and to analyze the waveform.

Theory:

- **PAM** is a modulation technique in which the amplitude of a train of pulses is varied according to the instantaneous value of the message signal.
- **Modulation:** Multiply the message signal by a pulse train.
- **Demodulation:** Recover the original signal using a low-pass filter or sample-and-hold reconstruction.

Mathematically:

$$s_{PAM}(t) = x(t) \cdot p(t)$$

where $x(t)$is the message signal and $p(t)$is the pulse train.

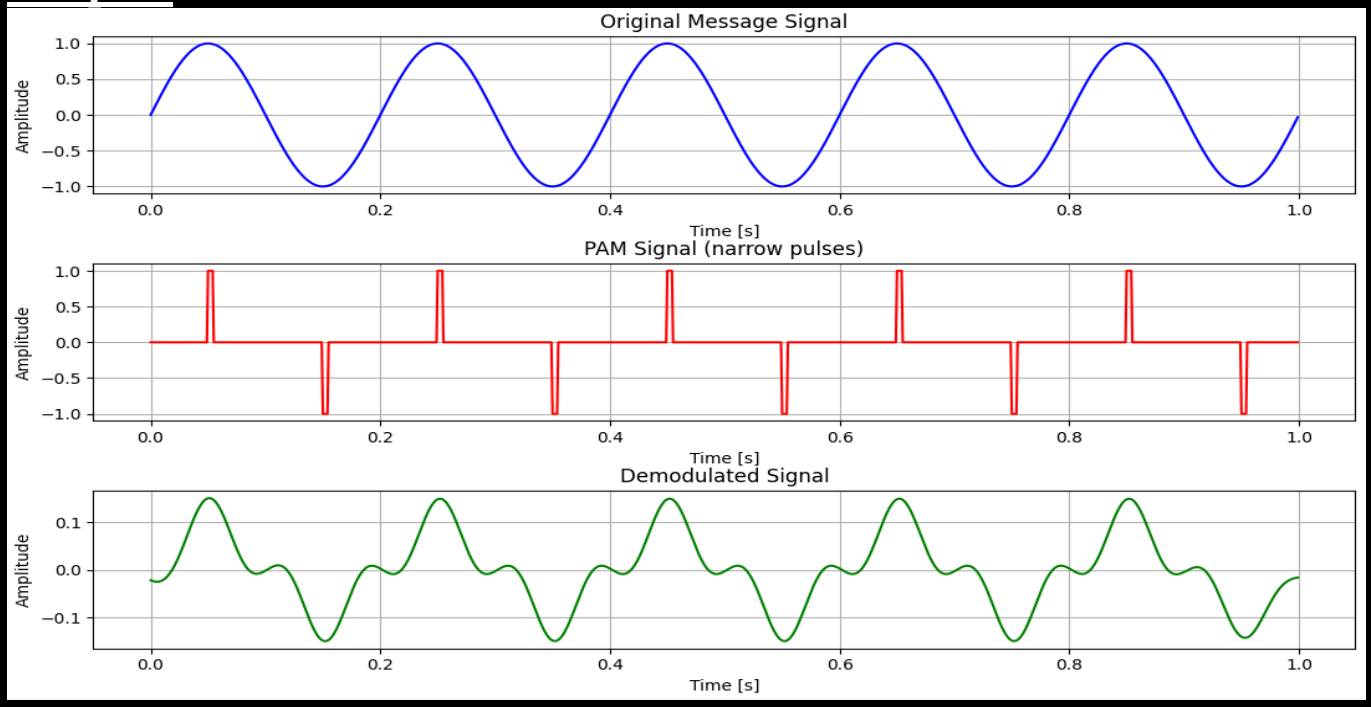
Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt
Fs = 1000            # Sampling frequency (Hz)
T = 1                # Duration of signal (seconds)
f_msg = 5            # Message frequency (Hz)
Ts = 0.05            # Pulse interval (seconds)
pulse_width = 0.005  # Pulse width in seconds
t = np.arange(0, T, 1/Fs)
x = np.sin(2 * np.pi * f_msg * t)
pam_signal = np.zeros_like(t)
pulse_samples = int(pulse_width * Fs)
for i in range(0, len(t), int(Ts * Fs)):
    end_idx = min(i + pulse_samples, len(t))  # Safe indexing
```

```
    pam_signal[i:end_idx] = x[i]
def butter_lowpass(cutoff, fs, order=5):
    nyq = 0.5 * fs
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    return b, a
def lowpass_filter(data, cutoff, fs, order=5):
    b, a = butter_lowpass(cutoff, fs, order=order)
    y = filtfilt(b, a, data)
    return y
cutoff_freq = f_msg * 3  # slightly higher to capture all frequency
components
demodulated_signal = lowpass_filter(pam_signal,
cutoff=cutoff_freq, fs=Fs)
plt.figure(figsize=(12, 8))
plt.subplot(3,1,1)
plt.plot(t, x, 'b')
plt.title('Original Message Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)
plt.subplot(3,1,2)
plt.plot(t, pam_signal, 'r')
plt.title('PAM Signal (narrow pulses)')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)
plt.subplot(3,1,3)
plt.plot(t, demodulated_signal, 'g')
plt.title('Demodulated Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
```

```
plt.grid(True)
plt.tight_layout()
plt.show()
error = np.max(np.abs(x - demodulated_signal))
print("PAM modulation and demodulation completed
successfully.")
print("Maximum reconstruction error:", error)
```

Output:



Result & Discussion:

- **Original Message Signal:** Smooth sinusoidal waveform.
- **PAM Signal:** The continuous sinusoid is sampled into pulses whose amplitude represents the signal.
- **Demodulated Signal:** After low-pass filtering, the original signal is closely reconstructed.

Discussion:

- PAM effectively converts an analog signal into a pulse-amplitude form.
- Demodulation using a low-pass filter retrieves the original waveform with minimal distortion.

Experiment No:-15

Title: Perform the DPCM modulation and demodulation using MATLAB/ Python.

Objective:

To simulate Differential Pulse Code Modulation (DPCM) and its demodulation using Python, and understand its efficiency in reducing redundancy in signal transmission.

Theory:

- **DPCM** encodes the **difference** between successive samples rather than absolute values.
- This reduces the data rate and exploits correlation between successive samples.
- **Encoding:**

$$d[n] = x[n] - x[n-1]$$

- **Decoding:**

$$\hat{x}[n] = \hat{x}[n-1] + d[n]$$

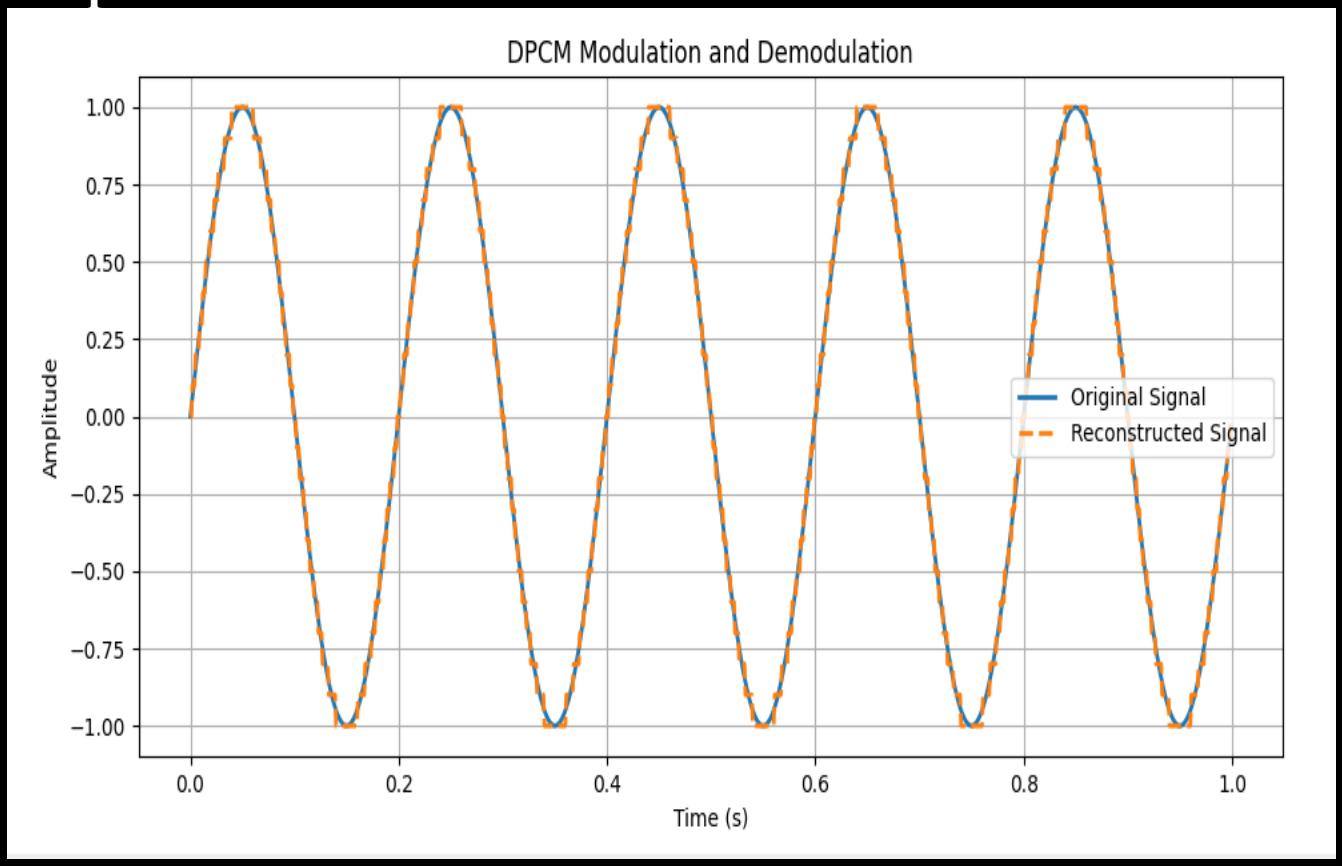
where $d[n]$is the differential signal, and $\hat{x}[n]$is the reconstructed signal.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
Fs = 1000            # Sampling frequency (Hz)
t = np.arange(0, 1, 1/Fs)  # Time vector (1 second)
A = 1                # Amplitude of the signal
f = 5                # Frequency of the signal (Hz)
x = A * np.sin(2 * np.pi * f * t)  # Original sinusoidal signal
delta = 0.1  # Quantization step size
x_hat = np.zeros(len(x))  # Predicted/reconstructed signal for
modulation
d = np.zeros(len(x))      # Quantized difference (DPCM signal)
x_hat[0] = x[0]
d[0] = x[0]
```

```
for n in range(1, len(x)):
    diff = x[n] - x_hat[n-1]       # Prediction error
    d[n] = delta * np.round(diff / delta)  # Quantize difference
    x_hat[n] = x_hat[n-1] + d[n]  # Update predicted signal
x_recon = np.zeros(len(x))
x_recon[0] = x[0]  # First sample directly transmitted
for n in range(1, len(d)):
    x_recon[n] = x_recon[n-1] + d[n]
plt.figure(figsize=(12, 6))
plt.plot(t, x, label='Original Signal', linewidth=2)
plt.plot(t, x_recon, '--', label='Reconstructed Signal', linewidth=2)
plt.title('DPCM Modulation and Demodulation')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()
plt.show()
mse = np.mean((x - x_recon)**2)
print(f"Mean Squared Error (MSE) between original and
reconstructed signal: {mse:.6f}")
```

Output :



Result & Discussion:

- **Original Signal:** Continuous sine wave.
- **DPCM Encoded Signal:** Shows the differences between consecutive samples, which are smaller in amplitude than the original signal.
- **Reconstructed Signal:** The original sine wave is recovered accurately by summing the differences.

Discussion:

- DPCM efficiently reduces the dynamic range of the transmitted data.
- Reconstruction is accurate if differences are encoded precisely.
- This technique is widely used in audio and speech compression to reduce bandwidth.